



Estruturas de Dados

Exame – Época Especial

Licenciatura em Engenharia Informática – Ano lectivo 2019/2020 – 60 min + 10 min.

Parte Teórica

Para todas as perguntas deste exame, apresente todos os passos intermédios relevantes, bem como uma brevíssima justificação/descrição daquilo que acontece em cada um desses passos. No entanto, não precisa de se repetir e voltar a dar explicações já dadas anteriormente.

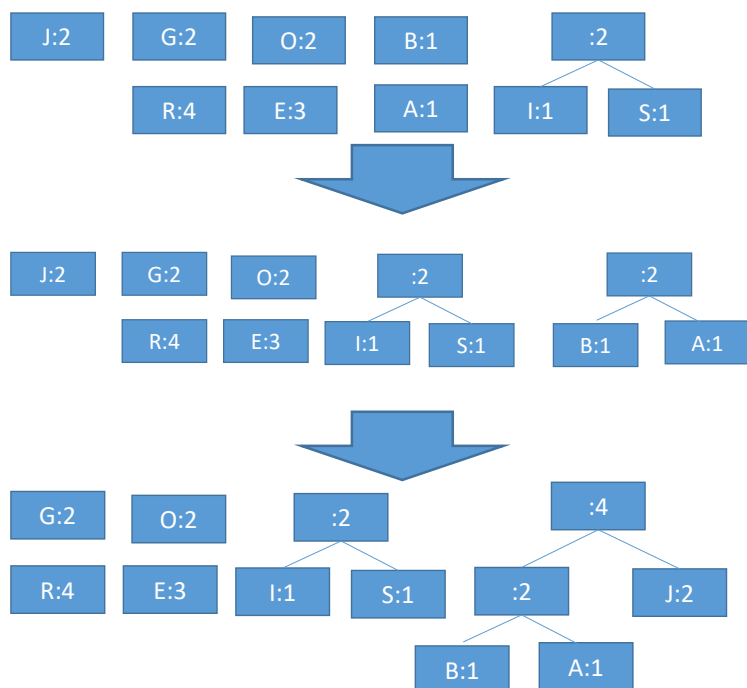
1 – [20%] Considere a string formada pela concatenação do seu primeiro nome, do seu apelido, e novamente do seu primeiro nome, sem espaços e escritos em maiúsculas. Por exemplo, no caso do regente da disciplina, esta string seria “JORGE BARREIROS JORGE” (no seu caso será obviamente uma string diferente). Construa uma árvore de Huffman adequada, indicando também qual a codificação de cada carácter de acordo com a árvore obtida. *Respostas que apresentem apenas a árvore resultante sem apresentação explícita dos passos de construção não são valorizadas.*

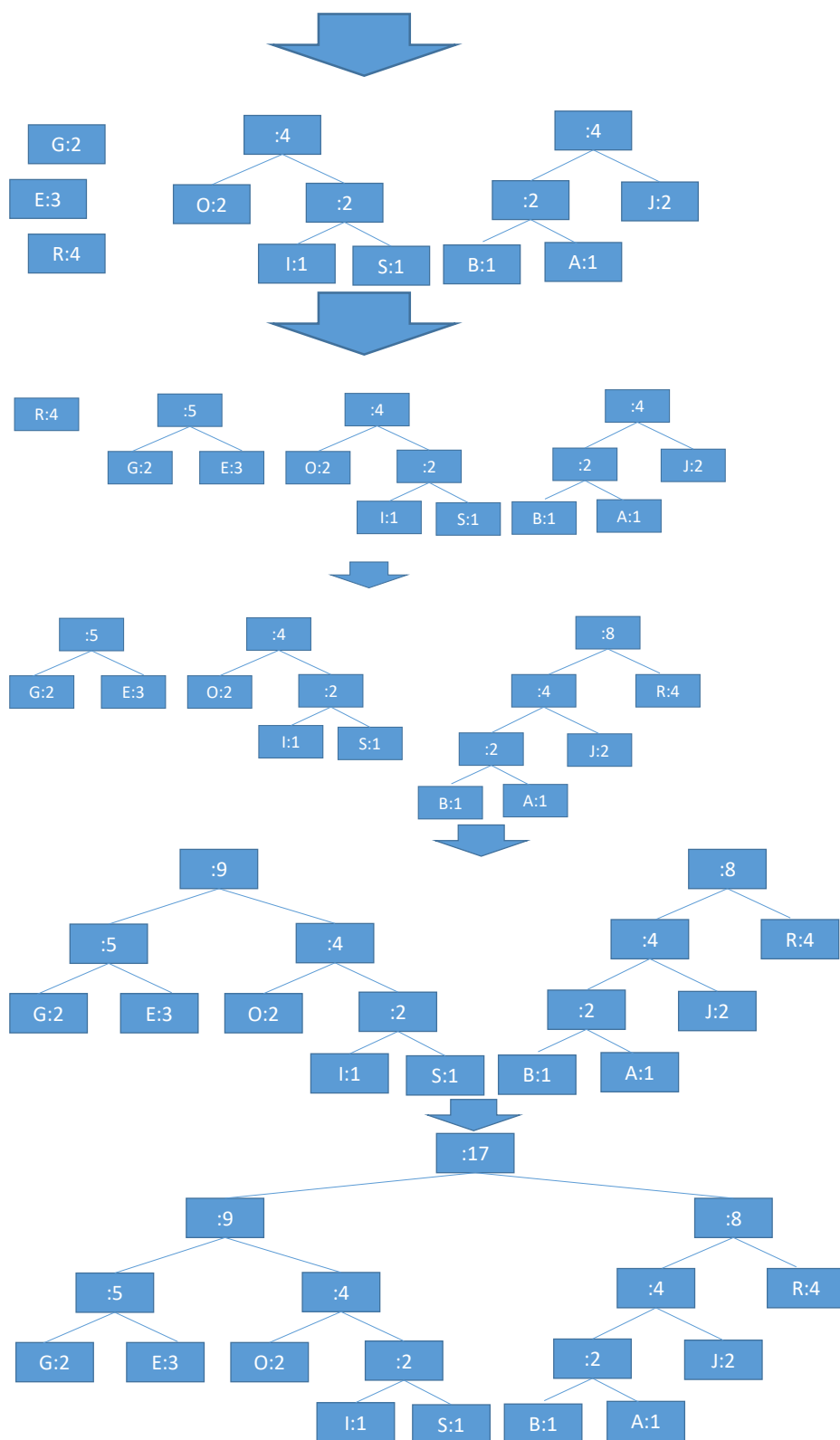
A pergunta será resolvida para a string indicada na pergunta, **no entanto, como é óbvio, deveria ser resolvida com o nome do próprio aluno, conforme indicado na pergunta.**

1º passo: determinar frequência de símbolos:



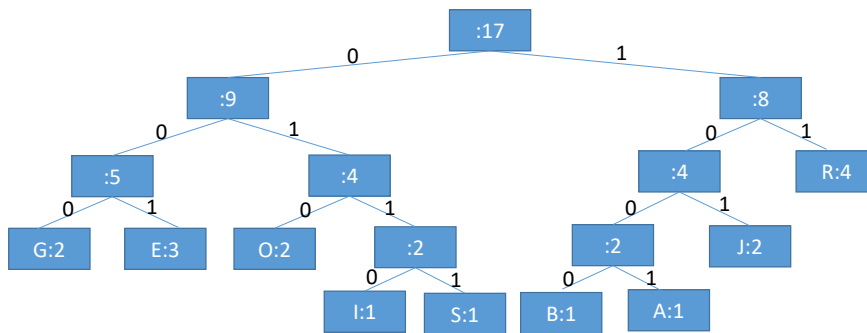
2º passo – construir a árvore agregando sempre os conjuntos com menos símbolos, desempatando de forma arbitrária. Aqui são apresentadas todos os conjuntos passo a passo, para efeitos de clarificação. Para efeitos de resposta, bastaria indicar em cada passo qual a árvore agregada.





3º passo: Extrair codificação:

Atribuir os valores 0 e 1 aos arcos da árvore e obter dessa forma o código correspondente a cada terminal. (como é evidente, na resposta seria tudo feito no mesmo diagrama, não seria necessário repetir o desenho).



Por fim, de acordo com o enunciado, indica-se a codificação de cada símbolo:

G:000
E:001
O:010
I: 0110
S:0111
B:1000
A:1001
J: 101
R:11

Termino assim, sem colocar a codificação da string inicial, porque li a pergunta com atenção e verifiquei que isso não é pedido, poupando assim tempo que poderei necessitar para responder ao resto do exame.

2-Para as perguntas seguintes, considere a seguinte associação entre letras e números

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	X	Y	W	Z
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

Considere as strings correspondentes ao seu primeiro nome e apelido. Para cada uma delas crie uma sequência de números fazendo a correspondência entre cada uma das letras para um número de acordo com a tabela apresentada. Por exemplo, “JORGE” corresponderia à sequência {10,15,18,7,5} e “BARREIROS” corresponderia à sequência {2,1,18,18,5,9,18,15,19}.

Pretendemos também que essas sequências tenham exatamente 10 valores. Replique o nome em questão o número de vezes suficiente para poder ter os 10 valores e.g. “JORGEJORGE”, “BARREIROSB” (neste caso basta acrescentar a primeira letra da 2ª repetição), resultando por fim nas sequências {10,15,18,7,5,10,15,18,7,5} e {2,1,18,18,5,9,18,15,19,2}. Estas sequências são designadas por S_{NOME} e S_{APELLIDO} , respetivamente, nas perguntas seguintes.

As sequências U_{NOME} e U_{APELLIDO} correspondem ao resultado de remover valores repetidos das sequências S_{NOME} e S_{APELLIDO} , deixando apenas a primeira ocorrência de cada valor repetido. Por fim, pretendemos que tanto U_{NOME} como U_{APELLIDO} tenham exatamente 10 valores. Para esse fim, deve, conforme seja necessário:

- Aumentar o número e valores (se forem menos do que 10), acrescentando em número suficiente os dígitos do seu número de estudante ao final da sequência, evitando no entanto acrescentar valores repetidos.
- Reduzir o número de valores (se forem mais do que 10), removendo valores do fim da sequência até restarem apenas 10.

Para o exemplo apresentado, se o número de estudante for 202012345, $U_{NOME} = \{10, 15, 18, 7, 5, \underline{2}, \underline{0}, \underline{1}, \underline{3}, \underline{4}\}$ e $U_{APELLIDO} = \{2, 1, 18, 5, 9, 15, 19, \underline{0}, \underline{3}, \underline{4}\}$, onde os valores sublinhados correspondem aos obtidos a partir do número de estudante.

2.1 – [20%] Considere que os valores da sua sequência U_{NOME} são inseridos sequencialmente na árvore AVL aqui apresentada. Pretendemos descobrir qual é a **primeira** dessas inserções que provoca a necessidade de rebalancear a árvore. Apresente:

- a árvore após a inserção mas antes desse rebalanceamento,
- indique qual o nodo desequilibrado,
- qual a regra de rebalanceamento a aplicar, e
- qual o a árvore resultante após a operação de rebalanceamento

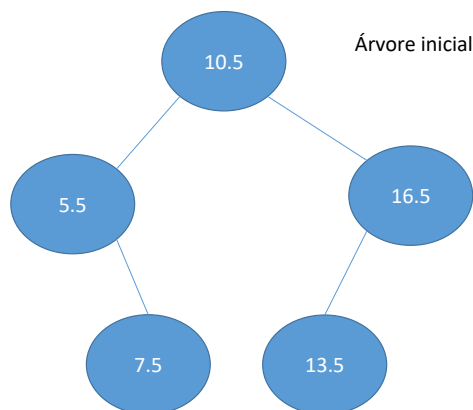
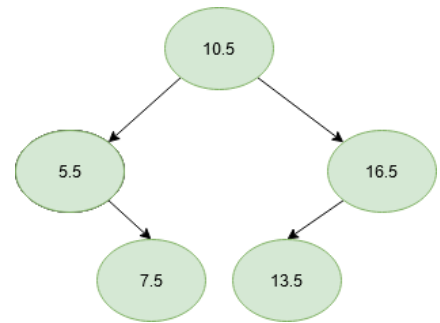
Respostas que não apresentem toda a informação aqui solicitada não serão valorizadas.

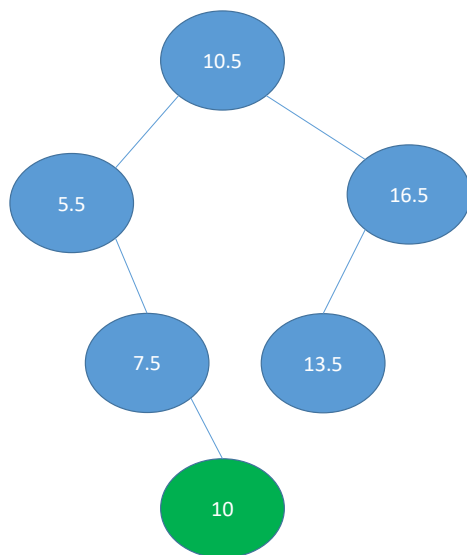
(não precisa de continuar e apresentar a inserção dos restantes elementos a partir deste ponto, estamos à procura apenas da primeira inserção que provoca um desequilíbrio).

Neste caso, $U_{NOME} = \{10, 15, 18, 7, 5, 2, 0, 1, 3, 4\}$. Tive o cuidado de confirmar que esta sequência foi bem construída e que não há valores repetidos....

Leio a pergunta com atenção, para perceber bem o que é solicitado e começo a considerar o

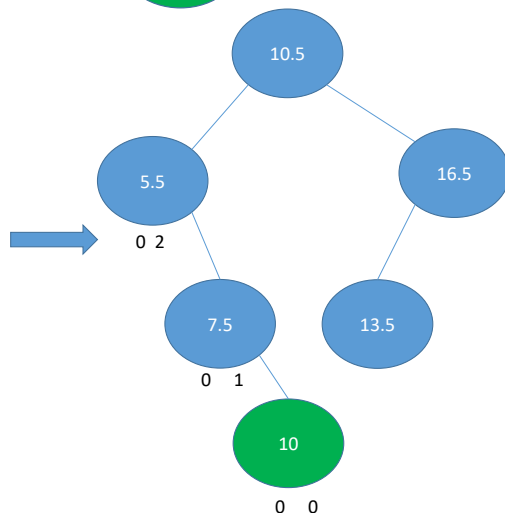
resultado da inserção dos valores de U_{NOME} **na árvore indicada** de forma a não perder logo à partida a maior parte do valor desta pergunta por não prestar atenção e não fazer aquilo que é pedido (esta árvore inicial deve estar cá por um motivo, não apenas porque sim...).



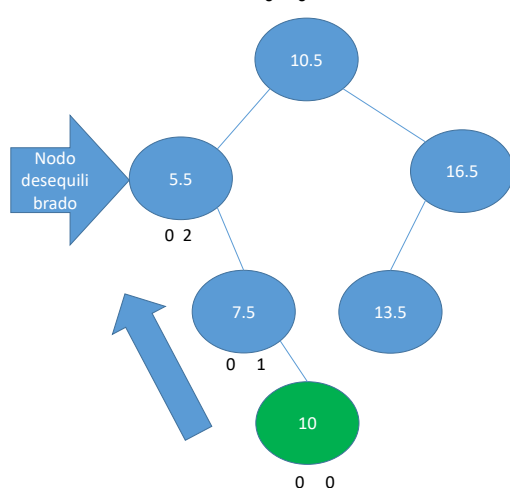


Inserção do 10.

A inserção começa na raiz. Como o valor a inserir é menor do que o valor da raiz (10.5), então o valor tem de ser inserido no lado esquerdo. Caso contrário, será no lado direito. O processo repete-se em cada nodo até ser atingido o local de inserção no fundo da árvore.



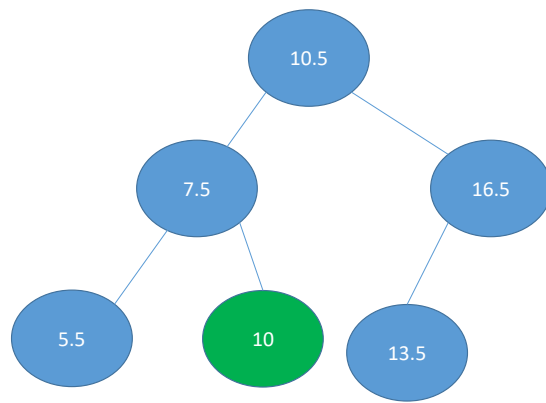
Após inserir o 10, verifica-se o equilíbrio de cada nodo no percurso inverso da inserção. Cada nodo está equilibrado se a diferença de altura entre as suas sub-árvores esquerda e direita não for superior a 1. Neste caso, verifica-se que o primeiro nodo desequilibrado (a partir do valor inserido) é o 5.5. Basta corrigir este desequilíbrio para equilibrar a árvore.



Para corrigir o desequilíbrio, vamos verificar o percurso percorrido até chegarmos ao nodo desequilibrado. Neste caso, verificamos que esse percurso é do formato:



O que significa que se trata de uma inserção EXTERNA. Neste caso, basta uma rotação, onde o nodo desequilibrado é rodado com o seu descendente ao longo deste percurso. Rodamos o 5.5 com o 7.5



Realizando a rotação, o 7.5 sobe e o 5.5 desce. Se o 7.5 tivesse descendentes esquerdos, estes passariam a ser descendentes direitos do 5.5. Tudo o resto mantém-se.

A árvore está equilibrada.

Verifico neste momento que a minha resposta inclui aquilo que é pedido, em particular:

- Já atingimos a primeira inserção que provoca um desequilíbrio
- a árvore após a inserção mas antes desse rebalanceamento, - está cá
- qual o nodo desequilibrado - é o 5.5
- qual a regra de rebalanceamento a aplicar - está descrita na resposta (correção de inserção externa)
- qual o a árvore resultante após a operação de rebalanceamento – também está cá.

Fico satisfeito por ter dado a resposta solicitada e não ter posto apenas algo como “desequilíbrio -> rotação” ou pior ainda não ter explicado nada. Fico também satisfeito por ter dado a resposta de acordo com o que é pedido, e também por ter construído a árvore pedida a partir da árvore indicada no enunciado. Após reflexão, chego à conclusão que não vale a pena gastar tempo a estudar e depois perder pontos porque não leio bem as perguntas.

Verifico também, com satisfação, que a pergunta indica que só precisamos de inserir valores até à primeira inserção que provoca um desequilíbrio, que foi o que já fiz. Sinto-me novamente satisfeito por ter prestado atenção aquilo que era pedido, porque assim evito perder tempo (e eventualmente pontos se me enganar) ao fazer coisas que não são pedidas. Constato novamente que dei por bem empregar o tempo “gasto” a ler bem a pergunta.

2.2 – [20%] Considere a sua sequência S_{NOME} . Indique qual o resultado da **inicialização** de uma *heap binária* com esses valores, de acordo com o processo de inicialização estudado nas aulas. *Como sabe, inicializar uma heap binária não é o mesmo que acrescentar os valores um a um. Este tipo de respostas não será valorizado.*

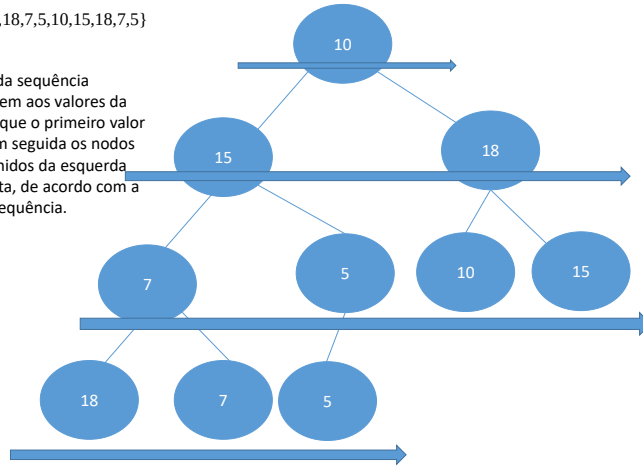
Verifico que a pergunta esclarece que inicializar uma heap binária não é o mesmo do que inserir os valores um a um. Este aviso é desnecessário mas simpático. Estou bem ciente da diferença, mas se não soubesse fazer o processo de inicialização porque não tinha estudado ou compreendido bem essa parte, poderia evitar perder tempo a resolver esta pergunta inserindo os valores um a um, porque é indicado claramente que esse tipo de respostas não é valorizado. O tempo seria melhor empregue a resolver a outras questões, em vez de me dedicar a uma resposta da qual não retirarei nenhuns pontos.

Mas, enfim, sei responder a esta. É muito fácil.

Após verificar que estou a usar a sequência correta (eu sei que não há problema em ter valores duplicados numa heap), sabendo que todas as heaps binárias são árvores binárias completas (ABC), construo a ABC resultante da interpretação implícita da árvore como a sequência de valores indicada

{10,15,18,7,5,10,15,18,7,5}

Os valores da sequência correspondem aos valores da árvore, em que o primeiro valor é a raiz e em seguida os nodos são preenchidos da esquerda para a direita, de acordo com a ordem da sequência.



Á árvore indicada não é ainda uma heap porque não respeita a propriedade de heap: o valor num nodo tem de ser menor ou igual do que todos os seus descendentes.

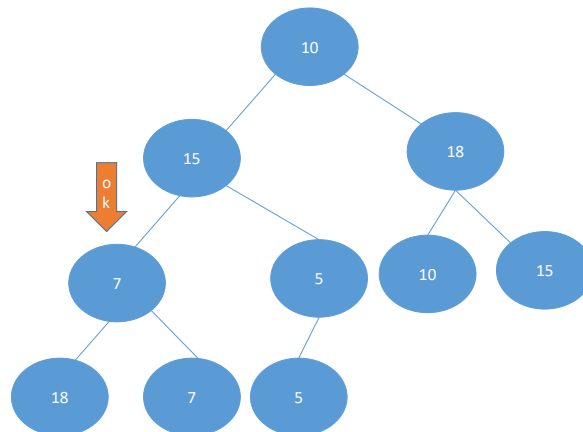
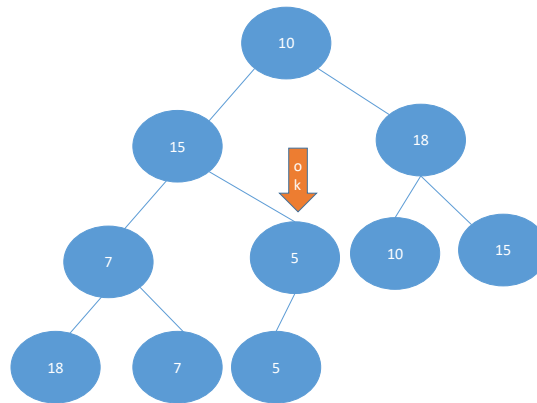
De forma a corrigir essa situação, vou aplicar o processo estudado nas aulas, porque sei que os valores não “voam” por magia entre nodos ou ao calhas para corrigir essa situação....

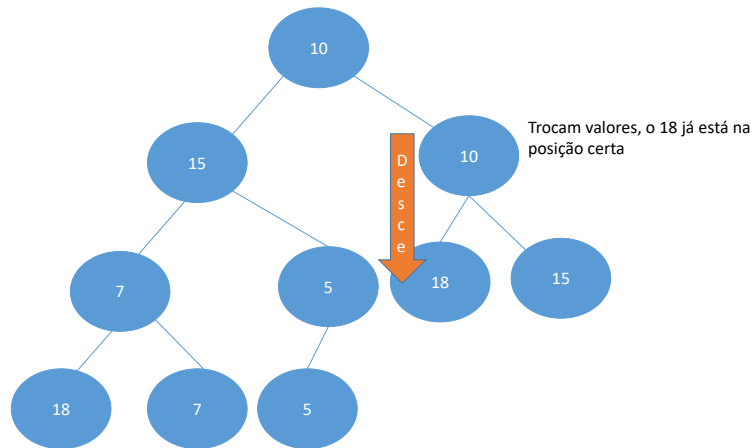
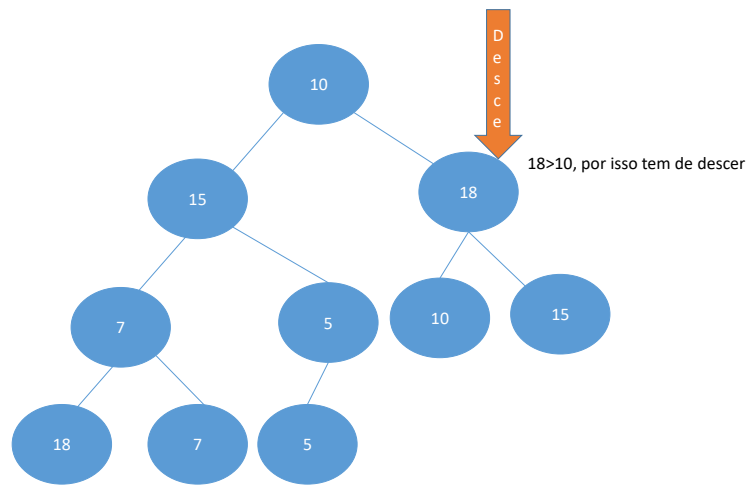
{10,15,18,7,5,10,15,18,7,5}

Começando no primeiro nodo não folha, verifico se este cumpre a propriedade de heap (ou seja, se é menor que os seus descendentes). Se isso não acontecer, o valor DESCE na direção do menor descendente, trocando com ele. Repito o processo até que o nodo que desce obedeça à propriedade de heap.

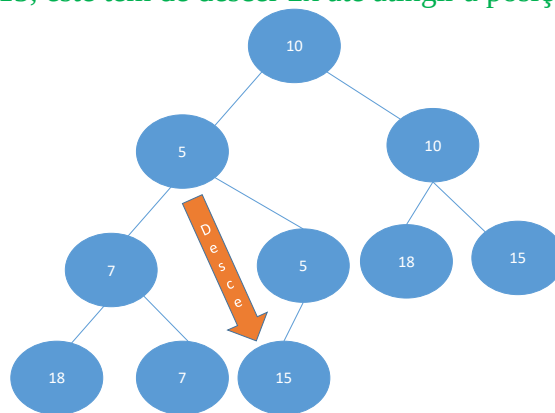
Neste caso, o 5 respeita a propriedade de heap, pelo que não é necessário fazer mais nada.

Continuamos na direção descendente da sequência (ou seja, vamos para o 7) e repetimos até chegarmos à raiz.

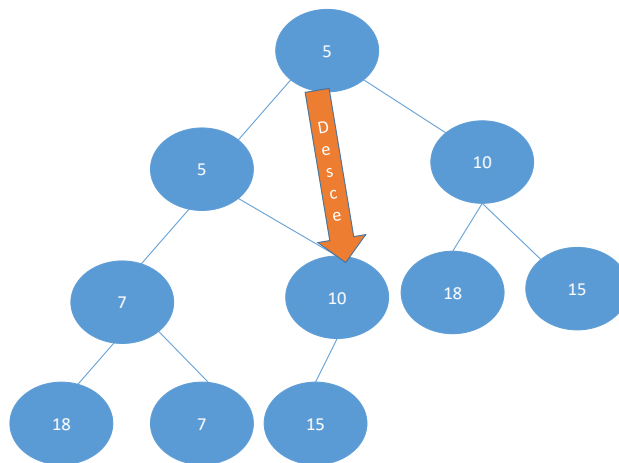




Repetindo o processo para o 15, este tem de descer 2x até atingir a posição correta

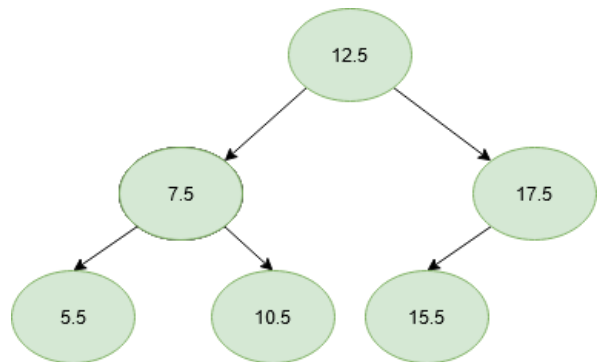


E o mesmo acontece com a raiz,



Isto termina o processo, e a heap binária está corretamente inicializada.

2.4 – [20%] Considere os primeiros 4 números da sua sequência S_{APELIDO} . Esta irá representar inserções e acessos a informação na splay tree aqui apresentada. A primeira vez que um valor aparece na sequência, representa uma inserção. Todas as outras instâncias desse número representam acessos. Por exemplo, se a sequência for $S_{\text{APELIDO}} = \{2, 1, 18, 18, 5, 9, 18, 15, 19, 2\}$, os números a considerar serão $\{2, 1, 18, \underline{18}\}$, onde os valores sublinhados correspondem a acessos para leitura e os não sublinhados a inserções. Indique qual o resultado da sequência de inserções/acessos resultante. *(conferir aviso a bold e sublinhado no início do enunciado)*



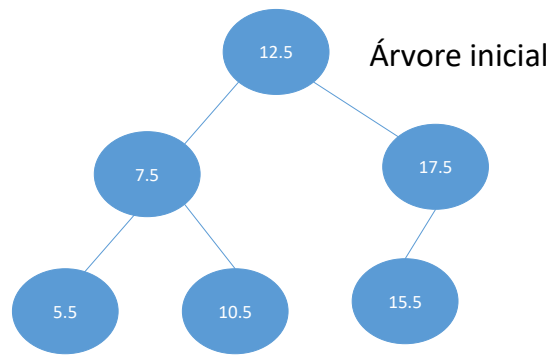
Neste caso, vamos usar a sequência $S_{\text{APELIDO}} = \{10, 15, 18, 7, 5, 2, 0, 1, 3, 4\}$, mas se não me chamasse Jorge Barreiros iria usar a sequência correspondente ao meu nome.

Leio a pergunta com atenção, para perceber bem o que é solicitado e começo a considerar o

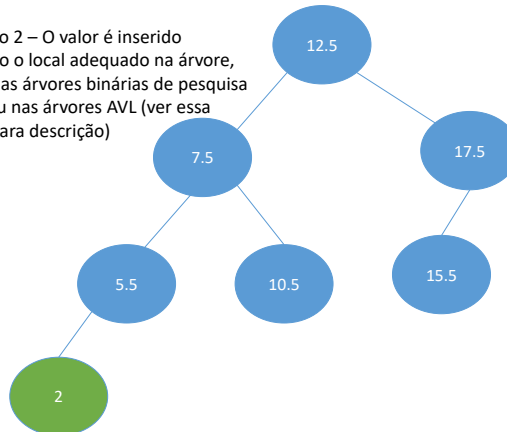
resultado da inserção dos valores **na splay tree indicada** de forma a não perder logo à partida a maior parte do valor desta pergunta por não prestar atenção e não fazer aquilo que é pedido (esta árvore inicial deve estar cá por um motivo, não apenas porque sim...o professor lá deve ter os seus motivos).

No nosso caso, a sequência é $\{2, 1, 18, 18\}$, sendo o segundo 18 correspondente a um acesso para leitura. Eu sei que TODOS os acessos nas splay trees (seja para leitura OU TAMBÉM para inserção) provocam um processo de rotação do nodo acedido para a raiz.

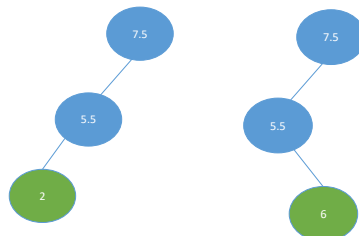
A árvore inicial é esta de acordo com o enunciado (penso de novo como é fácil não perder pontos simplesmente prestando atenção às perguntas).



Inserção do 2 – O valor é inserido procurando o local adequado na árvore, tal como nas árvores binárias de pesquisa normais ou nas árvores AVL (ver essa resposta para descrição)



Segue-se o processo que leva o nodo para a raiz. Considera-se o percurso correspondente, e verificamos em qual das duas situações seguintes nos encontramos (onde o nodo verde é o nodo a subir) – consideramos claro tb o caso simétrico

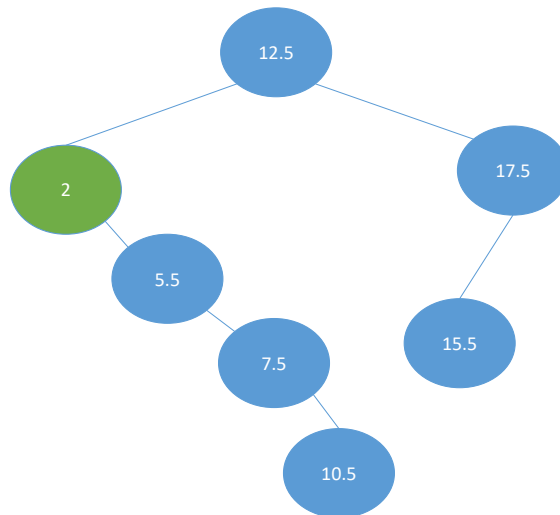


No caso da esquerda, é feita uma primeira rotação entre os dois nodos acima do valor acedido (neste caso, 7.5 e 5.5), sendo em seguida feita uma rotação entre o nodo que desceu (7.5) com o nodo acedido.

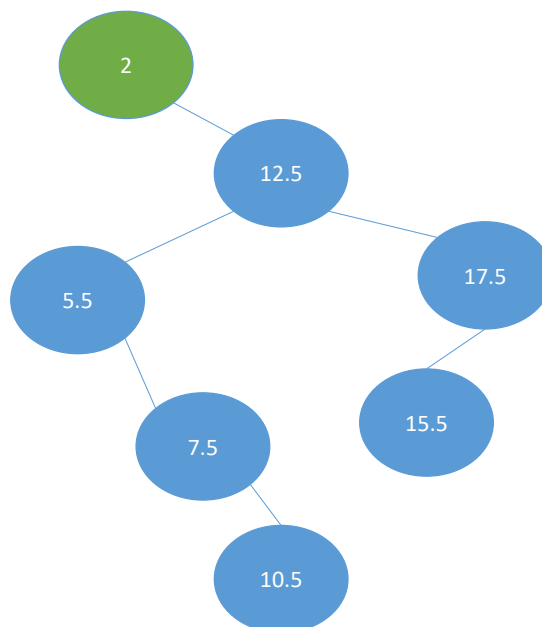
No caso da direita, as rotações são feitas de “baixo para cima”. O nodo acedido é primeiro rodado com o seu ascendente, e depois com o outro nodo.

Caso o nodo acedido seja diretamente descendente da raiz, é simplesmente rodado com a raiz.

Aplicando estas regras a este caso específico, verificamos que estamos na situação da esquerda (zig-zig) e obtemos

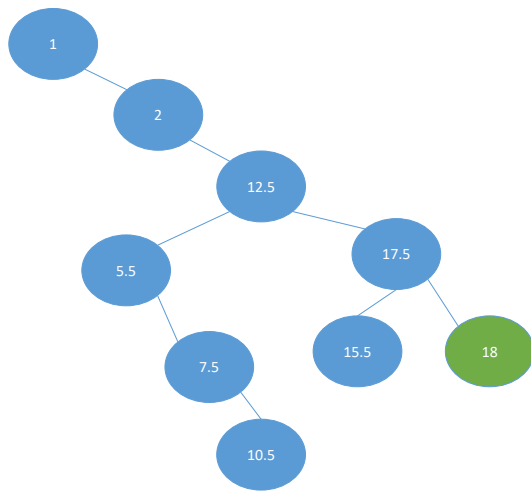
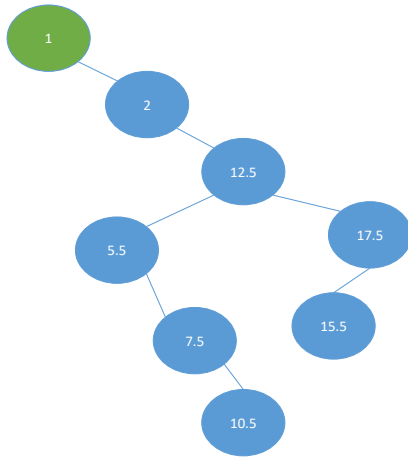
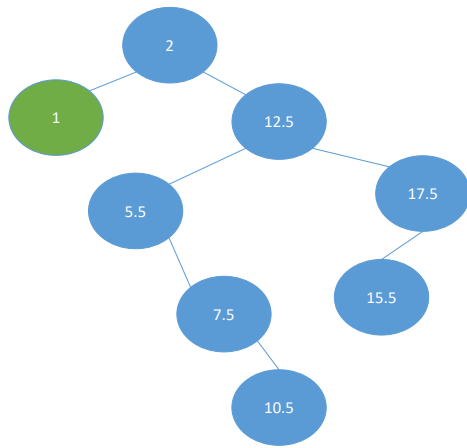


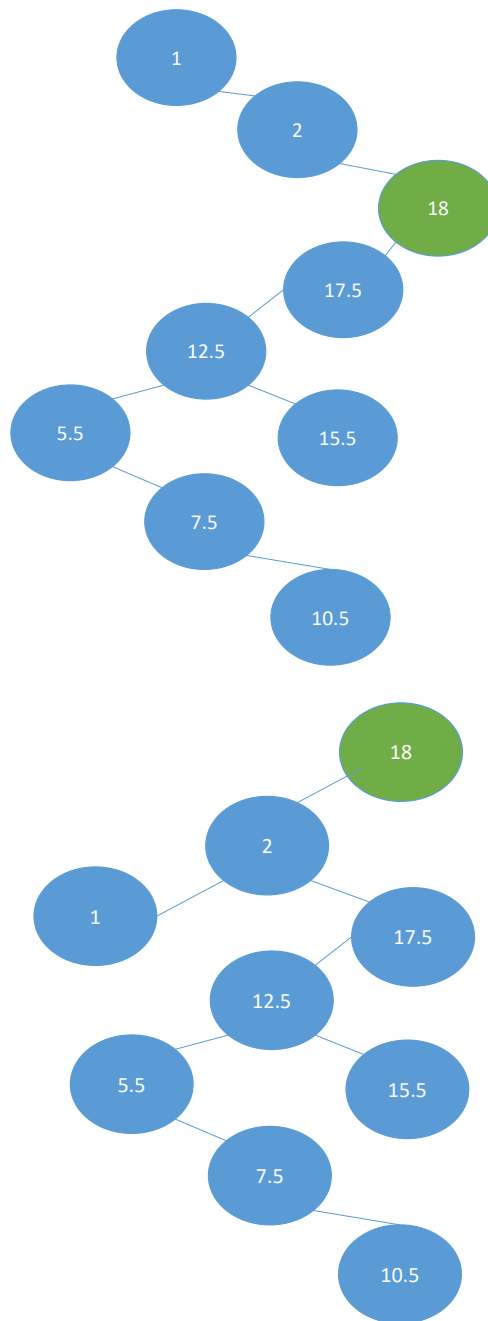
Roda-se agora com a raiz



Constato que toda aquela prática a treinar rotações compensou!

Inserido o 2, segue-se a inserção do 1 e do 18.





Por fim, o 2º acesso ao 18 é uma leitura. Tenho sorte, porque o 18 já está na raiz e portanto não há qualquer mudança da árvore. Terminamos aqui a nossa resposta. Esta pergunta é a que mais tempo demora a resolver, mas dado que não perdi tempo desnecessário nas outras perguntas, pratiquei e não preciso de estar a consultar apontamentos, o tempo chega e sobra.

2.3 – **[20%]** Considere a sua sequência U_{NOME} . Indique qual o resultado de introduzir esses valores numa tabela *hash* inicialmente vazia (~~com dimensão igual ao dobro do tamanho da sequência em questão~~) e função de *hash* $H(X)=2*X$, numa tabela com sondagem linear e dimensão 16. (*conferir aviso a bold e sublinhado no início do enunciado*)

ouvi a retificação da pergunta feita durante o exame e usei a dimensão 16. Mas podia ter usado dimensão 20, sem qualquer problema. A sequência é $U_{\text{NOME}} = \{10, 15, 18, 7, 5, 2, 0, 1, 3, 4\}$.

Para calcular a posição na tabela, calculamos o valor de hash e aplicamos o resto da divisão pela dimensão da tabela. No caso de colisões, elas são resolvidas procurando a próxima posição livre

(somando 1 à posição inicial sucessivamente até que se encontre um espaço vazio). Vou fazer uma tabela para ser mais fácil organizar a resposta.

X	H(X)	H(X)%16	Colisão?	Posição
10	20	4	N	4
15	30	14	N	14
18	36	4	S	5
7	14	14	S	15
5	10	10	N	10
2	4	4	S	6
0	0	0	N	0
1	2	2	N	2
3	6	6	S	7
4	8	8	N	8

Depois de verificar as contas, construo a tabela, tendo o cuidado de considerar o índice da primeira posição 0 (e não 1):

índice	Valor
0	0
1	
2	1
3	
4	10
5	18
6	2
7	3
8	4
9	
10	5
11	
12	
13	
14	15
15	7

E pronto, já está!